

3-2 · Pandas를 활용한 데이터 분석 · 정리·회고 0.5H

정리·회고

분석 문법을 한 장으로, 산출물은 파일로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

30분 뒤에는

- ① 분석 문법 열 가지를 「하려는 일 → 코드 → 주의」로 정리한다
- ② orders_enriched.parquet 가 1015행 · region 결측 0 인지 확인한다
- ③ 자주 겪는 오류 네 가지의 원인과 해결을 안다
- ④ 3-3 DuckDB 의 사전 준비(pip install duckdb)를 확인한다

체크리스트에서 비는 칸은 심화 과제로 이어 갑니다

분석 문법 — 집계 · 결합 · 교차표

하려는 일	코드	주의
그룹 집계	agg(새이름=(열, 함수))	명명 집계로 쓴다
여러 기준	groupby([a, b]) .reset_index()	인덱스를 열로 내린다
결합	merge(how='left', validate='many_to_one')	전후 행 수 비교 필수
키 중복 확인	b['k'].duplicated().sum()	merge 전에 한 줄
교차표	pivot_table(index, columns, values, aggfunc)	총합이 원본과 같은지

merge 전에 b['k'].duplicated().sum() 한 줄 — 대부분의 사고를 여기서 막는다

분석 문법 — 재구조화 · 시계열 · 통계

하려는 일	코드	주의
넓게 ↔ 길게	<code>melt() · pivot()</code>	사람은 넓게, 프로그램은 길게
날짜 인덱스	<code>set_index('d').sort_index()</code>	sort 를 빼지 말 것
시간 단위로 접기	<code>resample('D' 'W' 'ME').sum()</code>	행이 접힌다
이동 평균	<code>rolling(7).mean()</code>	행 수 유지, 앞쪽 NaN
기초 통계	<code>describe() · quantile()</code>	평균과 중앙값을 함께

resample 은 접고 rolling 은 유지한다 — 행 수를 세어 보면 차이가 바로 드러난다

orders_enriched.parquet — 3-3 에서 이어 쓸 준비

폴더	cd ~/data-eng/week3
파일	ls -lh *.parquet *.csv
행/열	pd.read_parquet('orders_enriched.parquet').shape
결측	d['region'].isna().sum() → 0
기준	1015행 · region 결측 0

1015행이고 region 결측이 0 이면 3-3 에서 이어서 쓸 준비가 된 것

체크리스트 — 스스로 확인해 보세요

- 1 `split-apply-combine` 세 단계를 말로 설명할 수 있다
- 2 명명 집계로 카테고리별 건수·매출·평균 단가를 산출했다
- 3 `groupby` 뒤 `reset_index()` 로 인덱스를 열로 내렸다
- 4 `merge` 에 `validate` 를 지정하고 전후 행 수를 비교했다
- 5 일부러 키를 중복시켜 행·합계가 부풀려지는 것을 봤다
- 6 `pivot_table` 교차표의 총합이 원본과 같음을 확인했다
- 7 `resample`(접힘)과 `rolling`(유지)의 차이를 행 수로 확인했다
- 8 `orders_enriched.parquet` 를 저장했다

증상으로 찾는 원인과 해결

증상	원인	해결
merge 후 행 수가 크게 불어남	오른쪽 표에서 조인 키 중복	duplicated() 점검 · validate 지정
Only valid with DatetimeIndex	인덱스가 날짜 타입이 아님	to_datetime → set_index 뒤 resample
열 이름이 두 겹 (MultiIndex)	agg 에 리스트· 딕셔너리로 넘김	명명 집계 agg(새이름=(열, 함수))
to_parquet 경고 · 이상한 열	groupby 인덱스가 남은 채 저장	reset_index() 또는 index=False

오류가 나지 않는 첫째가 가장 위험하다 — len() 비교 한 줄이 잘못된 보고를 막는다

자주 나오는 질문 네 가지

- 1 groupby 와 pivot_table 은 뭐가 다른가요?**
하는 일은 비슷 — groupby 는 세로로 길고, pivot_table 은 두 축 교차표로 넓게 나온다
- 2 merge 와 join 은요?**
join 은 인덱스 기준으로 붙이는 축약형 — 열을 키로 쓸 때는 merge
- 3 resample 과 groupby(pd.Grouper) 는 같은가요?**
거의 같다 — 다른 열로도 함께 묶어야 하면 Grouper
- 4 행이 들어났는지 매번 확인해야 하나요?**
네 — len() 비교 한 줄이면 되고, 이 습관 하나가 잘못된 보고를 막는다

다음 회차 예고

3-3 · DuckDB를 이용한 로컬 대용량 데이터 처리

01

이론 1H

DuckDB 개요와
열 지향 저장

02

실습 1H

파일 직접 질의
CSV · Parquet

03

실습 1.5H

COPY 변환 ·
크기·속도 실측

04

정리 0.5H

핵심 요약 ·
체크리스트 점검

준비: venv 에서 `pip install duckdb` · 디스크 여유 1GB 이상(100만 행 실습 파일) · 오늘 저장한 `orders_enriched.parquet`

3-2 · 정리·회고 0.5H 마무리

다음 회차는 3-3 DuckDB

서버 없이 내 노트북에서 100만 행을 SQL 로 다루고, CSV 와 Parquet 의 속도·크기 차이를 실측합니다

✓ DuckDB 와 Pandas 의 역할 차이

✓ 행 지향 vs 열 지향 저장

✓ CSV·Parquet 파일을 적재 없이 바로 질의

✓ COPY 로 압축 Parquet 변환 · 실측 비교

준비할 것

pip install duckdb · 디스크 1GB 여유